

# Flowra Sistem Mimarisi

## Teknik Altyapı ve Yazılım Tasarımı Başvuru Belgesi

Bu belge, Flowra'nın teknik mimarisini, veri akışını, güvenlik modelini ve entegrasyon noktalarını açıklar. Sistem yöneticileri, teknik danışmanlar ve uygulamayı derinlemesine anlamak isteyen yöneticiler için hazırlanmıştır.

## İçindekiler

- [Genel Bakış](#)
- [Beş Katmanlı Mimari](#)
- [Temel Teknoloji Bileşenleri](#)
- [Veri Akışı](#)
- [Güvenlik Mimarisi](#)
- [Veritabanı Yapısı — 38+ Tablo](#)
- [Entegrasyon Noktaları](#)
- [Otomatik Görevler \(Cron\)](#)
- [Çok Kiracılı Yapı](#)
- [Sistem Sağlığı ve İzleme](#)

## 1. Genel Bakış

Flowra, Türk küçük ve orta ölçekli işletmeleri (KOBİ) için tasarlanmış, çok kiracılı (multi-tenant) bir Finansal İşletim Sistemi'dir. Tek bir uygulama örneği üzerinde birden fazla bağımsız şirket çalışabilir; her şirket yalnızca kendi verilerine erişebilir.

Flowra iki temel ilke üzerine inşa edilmiştir:

**İlke 1 — Teknik bilgi gerektirmeyen muhasebe:** İşletme sahipleri ve çalışanları muhasebe terminolojisi bilmeden satış, masraf ve stok kaydedebilir. Sistem, bu operasyonel verileri otomatik olarak çift taraflı muhasebe kayıtlarına dönüştürür.

**İlke 2 — Türk mevzuatına tam uyum:** TTK, GVK, VUK ve KDV mevzuatına ilişkin hesaplamalar ve kontroller sistem içine gömülüdür; elle hesaplama gerektirmez.

## Teknik Özet

| Özellik              | Detay                               |
|----------------------|-------------------------------------|
| Mimari               | Çok Kiracılı SaaS (Multi-tenant)    |
| Frontend             | Next.js 14 (App Router)             |
| Backend              | Supabase (Postgres 15 + Auth + RLS) |
| Çalışma Ortamı       | Vercel Edge Network                 |
| Test Kapsamı         | 1.575 otomatik test (Vitest)        |
| Veritabanı Tabloları | 38+ tablo                           |
| Dil                  | TypeScript (tüm katmanlar)          |

## 2. Beş Katmanlı Mimari

Flowra'nın yazılım tasarımı beş katmandan oluşur. Her katman yalnızca bir altındaki katmanla doğrudan etkileşir; katmanlar arası kısa devre yapılmaz.

|                                                                           |                            |                              |                |
|---------------------------------------------------------------------------|----------------------------|------------------------------|----------------|
| KATMAN 5 – SUNUM KATMANI                                                  |                            |                              |                |
| CEO Komuta Merkezi                                                        | CFO Merkezi                | OPS Merkezi                  | Ortak Yönetimi |
| Kullanıcının doğrudan gördüğü ve etkileşime girdiği ekranlar              |                            |                              |                |
| KATMAN 4 – ZEKİ KATMAN (Intelligence Layer)                               |                            |                              |                |
| Durum Motoru (StatusEngine)                                               | Uyarı Motoru (AlertEngine) | Tahmin Motoru (ForecastEng.) | Risk Motoru    |
| İş kuralları, anormallik tespiti, proaktif bildirimler                    |                            |                              |                |
| KATMAN 3 – UYGULAMA KATMANI (Application Layer)                           |                            |                              |                |
| FinanceService                                                            | PCLEEngine                 | SimulationService            | WorkflowEng.   |
| Karmaşık iş mantığı, hesaplamalar, çok adımlı süreçler                    |                            |                              |                |
| KATMAN 2 – MUHASEBE DOĞRULUK KATMANI (Accounting Core)                    |                            |                              |                |
| GeneralLedger                                                             | JournalEntries             | TrialBalance                 | Period Close   |
| Çift taraflı kayıt garantisi, DR=CR denge kontrolü                        |                            |                              |                |
| KATMAN 1 – VERİ KATMANI (Data Layer)                                      |                            |                              |                |
| Supabase PostgreSQL 15 + Row Level Security<br>+ Supabase Auth + Realtime |                            |                              |                |
| Ham veri depolama, kimlik doğrulama, yetkilendirme                        |                            |                              |                |

## Katman 5 – Sunum Katmanı

Kullanıcıların gördüğü tüm ekranlar bu katmanda bulunur. Next.js App Router kullanılarak inşa edilmiştir. Sayfa bileşenleri iki türdedir:

- **Server Components:** Sayfa ilk yüklenirken sunucuda çalışır; SEO ve hız için optimize edilmiştir. Veritabanı sorguları doğrudan sunucuda çalışır.
- **Client Islands:** Etkileşimli öğeler (formlar, grafikler, açılır paneller) tarayıcıda çalışır. TanStack Query ile ön bellek yönetimi sağlanır.

Bu hibrit yapı, hem hızlı sayfa yükleme hem de zengin kullanıcı deneyimi sağlar.

## Katman 4 — Zeki Katman

Durum motorları ve uyarı sistemleri bu katmanda çalışır. Belirli koşullar sağlandığında otomatik bildirim üretir, risk skoru hesaplar ve yöneticileri proaktif olarak uyarır.

- **StatusEngine:** Her şirketin cari finansal durumunu (sağlıklı / dikkat / kritik) değerlendirir.
- **AlertEngine:** 13 önceden tanımlı kural setine göre uyarı üretir (vadesi geçen alacak, nakit tükenmesi, vb.).
- **ForecastEngine:** Geçmiş veri örüntülerine dayanarak 30-90 günlük nakit akışı tahmini üretir.
- **RiskEngine:** Ortak finansmanı dahil 6 boyutlu risk skoru hesaplar; A-F harf notu atar.

## Katman 3 — Uygulama Katmanı

Karmaşık iş mantığı bu katmanda işlenir:

- **FinanceService:** KDV hesaplamaları, kurumlar vergisi tahmini, nakit akışı analizi.
- **PCLEngine:** Ortak sermaye ve borç motoru; iki fazlı geri ödeme şelalesi, kâr dağıtımı hesaplamaları.
- **SimulationService:** "Ya olsaydı?" senaryo analizleri; büyüme varsayımları, gider değişkenleri.
- **WorkflowEngine:** Onay süreçleri; masraf onayı, dönem kapanışı, ortak işlemleri.

## Katman 2 — Muhasebe Doğruluk Katmanı

Her finansal işlem bu katmandan geçer. Bu katmanın temel görevi, çift taraflı kayıt dengesini hiçbir zaman bozmamaktır.

- **GeneralLedger:** Tüm hesapların bakiyelerini hesaplar ve ön belleğe alır.
- **JournalEntries:** Her operasyonel olayı otomatik muhasebe fişine dönüştürür.
- **TrialBalance:** Dönem sonu mizan hesaplaması ve denge doğrulaması.
- **PeriodClose:** Dönem kapatma süreci; kâr aktarımı, kilit mekanizması.

## Katman 1 — Veri Katmanı

Tüm verilerin kalıcı olarak depolandığı katmandır. Supabase platformu şu hizmetleri sağlar:

- **PostgreSQL 15:** İlişkisel veritabanı; ACID uyumlu işlemler.
  - **Row Level Security (RLS):** Her tablo düzeyinde otomatik şirket izolasyonu.
  - **Supabase Auth:** Kullanıcı kimlik doğrulama; e-posta/şifre ve SSO desteği.
  - **Realtime:** WebSocket üzerinden anlık veri güncellemesi (dashboard KPI'ları).
-

### 3. Temel Teknoloji Bileşenleri

#### 3.1 Next.js 14 App Router

Next.js, uygulamanın hem kullanıcı arayüzünü hem de API katmanını sunar.

| Özellik           | Kullanım                                           |
|-------------------|----------------------------------------------------|
| App Router        | Sayfa yönlendirme ve düzen yönetimi                |
| Server Components | Veri çekme, ilk sayfa render                       |
| Route Handlers    | REST API endpoint'leri ( <code>/api/**</code> )    |
| Middleware        | Auth kontrolü, rol doğrulama, kilitli dönem engeli |
| Server Actions    | Form işlemleri, veri mutasyonları                  |

#### 3.2 Supabase

Supabase, açık kaynaklı bir Firebase alternatifidir. Postgres veritabanı, kimlik doğrulama, dosya depolama ve gerçek zamanlı veri yayını birleştirilmiş tek bir platformda sunulur.

| Özellik        | Kullanım                             |
|----------------|--------------------------------------|
| Postgres 15    | Tüm iş verileri                      |
| Auth           | Kullanıcı oturumları, JWT token'ları |
| RLS Policies   | Şirket bazlı veri izolasyonu         |
| Storage        | Logo ve belge yükleme                |
| Realtime       | Dashboard anlık güncelleme           |
| Edge Functions | Ağır arka plan hesaplamaları         |

#### 3.3 TanStack Query

İstemci tarafında sunucu verilerinin yönetimi için kullanılır. Şu avantajları sağlar:

- Otomatik veri yeniden çekme (stale-while-revalidate)
- Ağ hatalarında otomatik tekrar deneme
- Sayfa geçişlerinde veri önbellekleme
- Eş zamanlı sorgu deduplication

#### 3.4 Vitest Test Çerçevesi

1.575 otomatik test, uygulamanın her yeni sürümde doğru çalıştığını garanti eder:

- **Birim testleri:** Her servis fonksiyonunun izole testi
- **Entegrasyon testleri:** Katmanlar arası veri akışı testi
- **Muhasebe invariant testleri:** DR=CR dengesi, TTK kısıtları, FIFO hesaplamaları

### 3.5 jsPDF

Markalı PDF üretimi için kullanılır. Şu belgeleri üretir:

- Proforma / teklif PDF'leri (müşteri logolu)
- Finansal tablo PDF'leri (bilanço, gelir tablosu)
- Mutabakat raporu PDF'leri
- Dönem kapanış belgeleri

### 3.6 Anthropic Claude API (Opsiyonel)

Yapay zeka destekli durum özetleri için kullanılır. `ANTHROPIC_API_KEY` ortam değişkeni tanımlı değilse bu özellik sessizce devre dışı kalır; uygulama çalışmaya devam eder. Claude API şunları üretir:

- CEO komuta merkezindeki doğal dil durum özetleri ("Bu ay satışlar %15 arttı, ancak tahsilat süresi uzuyor...")
- CFO için aylık anormallik tespiti ve yorum

---

## 4. Veri Akışı

### 4.1 Operasyonelden Finansale Veri Yolu

Bir satış kaydedildiğinde veri şu yolu izler:

### 1. KULLANICI GİRİŞİ

Kullanıcı Ticari Hub > Satışlar > Yeni Satış formunu doldurur  
(müşteri adı, ürün, miktar, birim fiyat, KDV oranı)

|



### 2. API ROUTE DOĞRULAMA

POST /api/sales

- Auth kontrolü: Geçerli oturum var mı?
- Rol kontrolü: Kullanıcının sales:write yetkisi var mı?
- Dönem kontrolü: Dönem kilitli mi? (kilitliyse 403 döner)
- Veri doğrulama: Zorunlu alanlar var mı? Tutar > 0 mı?

|



### 3. OPERASYONELİ KAYIT

sales tablosuna yeni satır eklenir

(company\_id, amount, vat\_amount, customer\_id, status='open')

|



### 4. OTOMATİK YEVMİYE FİŞİ

GL modu shadow ise bu adım atlanır.

GL modu parallel veya gl\_primary ise:

JournalEntryService.createSaleJournal() çağrılır

Üretilen fiş satırları:

DR 120 Alıcılar = satış\_tutarı + KDV

CR 600 Satışlar = KDV hariç satış tutarı

CR 391 Hesap. KDV = KDV tutarı

+ COGS fişi (stok varsa):

DR 620 SMM = FIFO maliyet

CR 153 Stok = FIFO maliyet

|



### 5. HESAP BAKİYELERİ GÜNCELLENİR

GeneralLedger.updateAccountBalance() çağrılır

Etkilenen hesapların önbellek bakiyeleri güncellenir

|



### 6. DENETİM GÜNLÜĞÜ

audit\_logs tablosuna kayıt eklenir

(user\_id, action='create', entity\_type='sale', payload, hash)

|



### 7. FİNANSAL TABLOLARA YANSIMA

Dashboard KPI'ları: TanStack Query ile otomatik revalidate

Gelir tablosu: 600 hesabının CR kalanı arttı

Bilanço: 120 hesabı arttı

Nakit akışı: Tahsilat bekleniyor

## 4.2 Tahsilat Akışı

Müşteri ödeme yapar → sale\_payment kaydı oluşturulur

- |
- └─> DR 102 Bankalar (gelen tutar)
- | CR 120 Alıcılar (alacak kapandı)
- |
- └─> Satış durumu: 'open' → 'paid' (kısmi veya tam)
- |
- └─> Nakit akışı tablosu güncellenir

## 4.3 Stok ve FIFO Akışı

Satın alma girilir (Operasyon > Stok > Satın Al)

- |
- └─> purchases tablosuna kayıt
- └─> stock\_lots tablosuna yeni lot (lot\_id, quantity, unit\_cost)
- └─> DR 153 Stok | CR 102 Bankalar (veya 320 Satıcılar)

Satış gerçekleşir

- |
- └─> stock\_lots'tan FIFO sırayla lot tüketilir
- | (en eski lot önce çıkar)
- └─> lot\_quantity azaltılır veya lot kapatılır
- └─> DR 620 SMM | CR 153 Stok (FIFO maliyet üzerinden)

## 5. Güvenlik Mimarisi

### 5.1 Çok Katmanlı Güvenlik Modeli

Flowra'da güvenlik dört bağımsız katmanda uygulanır. Bir saldırganın tüm dört katmanı aşması gerekir; herhangi birinde durur.

KATMAN A – KİMLİK DOĞRULAMA (Authentication)  
Supabase Auth; JWT token her istekte doğrulanır  
Geçersiz veya süresi dolmuş token → 401 Unauthorized  
|  
▼  
KATMAN B – ROL YETKİLENDİRMESİ (Authorization)  
API route'da rol kontrolü (Admin / Manager / Viewer)  
Yetersiz rol → 403 Forbidden  
(Bu kontrol, veritabanı sorgusundan ÖNCE çalışır)  
|  
▼  
KATMAN C – SATIR DÜZEYİ GÜVENLİK (Row Level Security)  
PostgreSQL içinde her tablo için RLS politikası  
Her sorgu otomatik olarak:  
WHERE company\_id = auth.jwt() -> 'company\_id'  
koşulıyla kısıtlanır  
Başka şirketin verisine erişim imkânsızdır  
|  
▼  
KATMAN D – DÖNEM KİLİDİ (Period Lock)  
Kilitlenmiş dönemler için middleware tüm yazma  
işlemlerini engeller; salt okunur API rotaları dışında  
hiçbir değişikliğe izin verilmez

## 5.2 Row Level Security Politikası

Her tablo için örnek RLS politikası:

```
-- sales tablosu için okuma politikası
CREATE POLICY "company_isolation_select"
ON sales FOR SELECT
USING (
  company_id = (auth.jwt() ->> 'company_id')::uuid
);

-- sales tablosu için yazma politikası
CREATE POLICY "company_isolation_insert"
ON sales FOR INSERT
WITH CHECK (
  company_id = (auth.jwt() ->> 'company_id')::uuid
);
```

Bu politikalar veritabanı motoru seviyesinde çalışır. Uygulama kodu yanlış yazılsa bile başka şirkete ait veri okunamaz veya yazılamaz.

## 5.3 Denetim Hash Zinciri

Her finansal aksiyon `audit_logs` tablosuna kaydedilir. Kayıtlar, önceki kaydın SHA-256 hash değerini içerir:

```
Kayıt N:
  content_hash = SHA256(action + entity_id + payload + timestamp)
  prev_hash   = content_hash değeri kayıt N-1'den

Kayıt N+1:
  content_hash = SHA256(action + entity_id + payload + timestamp)
  prev_hash   = content_hash değeri kayıt N'den
```

Bu zincir yapısı sayesinde geçmiş bir kaydın değiştirilmesi tespit edilebilir. Zincir doğrulaması şu endpoint ile çalıştırılır:

```
GET /api/admin/audit-chain-verify?from=YYYY-MM-DD&to=YYYY-MM-DD
```

## 5.4 Ortam Değişkenleri Güvenliği

| Değişken                               | Önem   | Açıklama                                                                                    |
|----------------------------------------|--------|---------------------------------------------------------------------------------------------|
| <code>SUPABASE_SERVICE_ROLE_KEY</code> | Kritik | Tüm RLS'yi atlar; yalnızca sunucu tarafında kullanılır, istemciye asla gönderilmez          |
| <code>CRON_SECRET</code>               | Yüksek | Cron job endpoint'lerini dışarıdan çağrılmaya karşı korur                                   |
| <code>ANTHROPIC_API_KEY</code>         | Orta   | AI özet özelliği için; yoksa özellik devre dışı kalır                                       |
| <code>RESEND_API_KEY</code>            | Orta   | E-posta bildirimleri için; yoksa e-posta sessizce devre dışı kalır                          |
| <code>ENABLE_SEED</code>               | Kritik | Üretim ortamında <code>false</code> olmalı; <code>true</code> ise demo verileri eklenebilir |
| <code>ENABLE_RESET</code>              | Kritik | Üretim ortamında <code>false</code> olmalı; <code>true</code> ise tüm veriler silinebilir   |

## 6. Veritabanı Yapısı — 38+ Tablo

Tablolar beş kategoride sınıflandırılır:

### 6.1 Finansal Tablolar

Bu tablolar günlük iş operasyonlarını tutar. Kullanıcılar bu tablolara veri girer.

| Tablo                      | Açıklama             | Temel Alanlar                                                  |
|----------------------------|----------------------|----------------------------------------------------------------|
| <code>sales</code>         | Satış kayıtları      | company_id, customer_id, amount, vat_amount, status, sale_date |
| <code>sale_payments</code> | Tahsilat kayıtları   | sale_id, amount, payment_date, payment_method                  |
| <code>expenses</code>      | Masraf kayıtları     | company_id, amount, category, status, paid_at                  |
| <code>purchases</code>     | Stok alımları        | company_id, product_id, quantity, unit_cost, total_amount      |
| <code>stock_lots</code>    | FIFO stok lotları    | product_id, quantity, unit_cost, remaining_qty, lot_date       |
| <code>customers</code>     | Müşteri kartları     | company_id, name, tax_id, contact_info                         |
| <code>products</code>      | Ürün/hizmet katalogu | company_id, name, unit_price, vat_rate, stock_tracking         |

## 6.2 Muhasebe Tabloları

Bu tablolar GL motoru tarafından yönetilir. Kullanıcı doğrudan düzenleyemez.

| Tablo                           | Açıklama               | Temel Alanlar                                                           |
|---------------------------------|------------------------|-------------------------------------------------------------------------|
| <code>journal_entries</code>    | Tüm muhasebe işleri    | account_code, debit, credit, source_type, source_id, voucher_no, period |
| <code>accounting_periods</code> | Dönem yönetimi         | company_id, period (YYYY-MM), is_locked, closed_at                      |
| <code>account_balances</code>   | Hesap bakiye önbelleği | account_code, period, balance (yeniden hesaplanabilir)                  |

## 6.3 PCLE Tabloları (Ortak Finansmanı)

| Tablo                               | Açıklama                    | Temel Alanlar                                                          |
|-------------------------------------|-----------------------------|------------------------------------------------------------------------|
| <code>partners</code>               | Ortak tanımları             | company_id, name, equity_share_pct                                     |
| <code>partner_finance_events</code> | Değiştirilemez olay günlüğü | partner_id, event_type, amount, created_at                             |
| <code>partner_loan_tranches</code>  | Kredi dilimleri             | partner_id, principal, interest_rate, maturity_date, remaining_balance |
| <code>partner_distributions</code>  | Kâr dağıtım kayıtları       | partner_id, amount, type (dividend/repayment), declared_at             |
| <code>partner_capital</code>        | Sermaye hareketleri         | partner_id, amount, transaction_type                                   |

## 6.4 Yönetişim Tabloları

| Tablo                                 | Açıklama                        | Temel Alanlar                                                    |
|---------------------------------------|---------------------------------|------------------------------------------------------------------|
| <code>workflow_instances</code>       | Onay süreçleri                  | type, status, submitted_by, approved_by, timeout_at              |
| <code>workflow_steps</code>           | Onay adımları                   | workflow_id, step_order, approver_role, action, acted_at         |
| <code>audit_logs</code>               | Denetim günlüğü                 | action, entity_type, entity_id, user_id, content_hash, prev_hash |
| <code>reconciliation_snapshots</code> | Mutabakat anlık görüntüleri     | period, data_json, signed_by, signed_at, content_hash            |
| <code>governance_snapshots</code>     | Yönetişim periyodik görüntüleri | period, snapshot_type, data_json                                 |
| <code>alert_rules</code>              | Uyarı kuralı tanımları          | rule_code, threshold, is_active                                  |
| <code>alert_instances</code>          | Tetiklenen uyarılar             | rule_code, triggered_at, dismissed_at, severity                  |

## 6.5 Konfigürasyon Tabloları

| Tablo                                 | Açıklama                    | Temel Alanlar                            |
|---------------------------------------|-----------------------------|------------------------------------------|
| <code>companies</code>                | Şirket tanımları            | name, tax_id, gl_mode, logo_url          |
| <code>company_members</code>          | Kullanıcı-Şirket ilişkileri | user_id, company_id, role                |
| <code>users</code>                    | Kullanıcı profilleri        | email, full_name, avatar_url             |
| <code>notification_preferences</code> | Bildirim tercihleri         | user_id, channel, event_type, is_enabled |

## 7. Entegrasyon Noktaları

### 7.1 Supabase REST API ve Realtime

Flowra, Supabase'in otomatik oluşturulan REST API'sini şu amaçlarla kullanır:

- Standart CRUD işlemleri (okuma/yazma/silme)
- Realtime kanallar: Dashboard KPI'ları anlık güncellenir; yeni satış kaydedildiğinde CEO ekranı otomatik yenilenir

## 7.2 Resend (E-posta Bildirimleri)

`RESEND_API_KEY` tanımlı olduğunda şu durumlarda e-posta gönderilir:

- Yeni kullanıcı davet edildiğinde (aktivasyon bağlantısı)
- Onay isteği oluşturulduğunda (onaylayacak kişiye)
- Onay tamamlandığında (isteği açana)
- Uyarı eşiği aşıldığında (kritik uyarılar)
- Dönem kapanış özeti (CFO'ya)

## 7.3 Anthropic Claude API (AI Özetler)

`ANTHROPIC_API_KEY` tanımlı olduğunda CEO komuta merkezinde doğal dil özetleri görünür. API yoksa ekran yalnızca sayısal KPI kartlarını gösterir; hiçbir fonksiyon bozulmaz.

Claude API çağrısı şu şekilde çalışır:

- Sunucu, şirketin son 30 günlük özetini JSON olarak hazırlar
- Claude API'ye gönderilir (maksimum maliyet kontrolü için token limiti uygulanır)
- Döndürülen Türkçe özet, CEO ekranının üst bandında gösterilir
- Yanıt 6 saat önbelleğe alınır; her sayfa yenilemede API çağrılmaz

## 7.4 Vercel Cron (Arka Plan Görevleri)

Dört otomatik görev Vercel'in cron altyapısında çalışır. Tetikleme, `CRON_SECRET` header'ı doğrulandıktan sonra yapılır (dışarıdan yetkisiz çağrıya karşı korunur).

## 8. Otomatik Görevler (Cron)

| Görev                          | Zamanlama          | Açıklama                                                                                                                                |
|--------------------------------|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Vadesi Geçen Güncelleme</b> | Her gece 02:00     | Ödeme tarihi geçmiş satışların durumunu <code>overdue</code> olarak işaretler; RECEIVABLE_30 ve RECEIVABLE_60 uyarılarını tetikler      |
| <b>Faiz Tahakkuku</b>          | Her gece 03:00     | Aktif ortak kredi dilimlerinin dönemsel faizini hesaplar; 780 ve 321/421 hesaplarına otomatik fiş yazar                                 |
| <b>İş Akışı Süresi Dolma</b>   | Her gece 04:00     | 48 saati aşmış onay bekleyen iş akışlarını otomatik olarak <code>expired</code> statüsüne çeker; ilgili kullanıcılara bildirim gönderir |
| <b>Yönetişim Snapshot</b>      | Her ayın 1'i 05:00 | Aylık yönetim anlık görüntüsü oluşturur; ortak pozisyonları, risk skorları ve bilanço özetini arşivler                                  |

Cron görevlerinin çalışma geçmişi `/dashboard/admin` > Sistem sekmesinden izlenebilir. Her görevin son çalışma zamanı, süresi ve sonucu (başarı/hata) görüntülenir.

## 9. Çok Kiracılı Yapı

### 9.1 Veri İzolasyonu

Her veritabanı tablosu `company_id` sütunu taşır. RLS politikaları, her kullanıcının yalnızca kendi şirketinin verilerine erişmesini otomatik olarak sağlar.

```
Kullanıcı A → Şirket X (RLS: company_id = X)
Kullanıcı B → Şirket Y (RLS: company_id = Y)
Kullanıcı C → Şirket X ve Z (çoklu şirket üyeliği)
```

### 9.2 Çoklu Şirket Desteği

Bir kullanıcı birden fazla şirkette üye olabilir. `company_members` tablosu kullanıcı-şirket-rol üçlüsünü tutar. Kullanıcı oturum açtığında, üyesi olduğu şirketler listelenir. Sidebar'daki şirket değiştirici (company switcher) ile şirket geçişi yapılır.

Şirket geçişi sırasında:

1. Yeni şirketin `company_id` JWT token'a yazılır
2. Tüm aktif sorgular iptal edilir
3. Yeni şirketin verileriyle sayfa yenilenir

### 9.3 GL Modu Per-Şirket

Her şirketin bağımsız bir GL modu vardır. Şirket A `gl_primary` modunda çalışırken şirket B `shadow` modunda kalabilir. Mod değişikliği yalnızca o şirketin verilerini etkiler.

## 10. Sistem Sağlığı ve İzleme

### 10.1 Health Endpoint

```
GET /api/health
```

Yanıt örneği:

```
{
  "status": "healthy",
  "timestamp": "2026-05-26T10:30:00Z",
  "checks": {
    "database": { "status": "ok", "latency_ms": 12 },
    "gl_mode": { "status": "gl_primary" },
    "open_period": { "status": "ok", "period": "2026-05" },
    "trial_balance": { "status": "balanced", "delta": 0.00 }
  }
}
```

## 10.2 Temel İzleme Metrikleri

### Vercel Dashboard:

- Function çalışma süresi (her API route için)
- Edge Network hata oranı
- Cron görev çalışma geçmişi

### Supabase Dashboard:

- Aktif bağlantı sayısı
- Sorgu gecikme ortalaması
- Veritabanı boyutu
- Auth kullanıcı sayısı

### Uygulama İzleme:

- `/api/admin/gl-readiness` : GL kapsam yüzdesi
- `/api/admin/audit-chain-verify` : Denetim zinciri bütünlüğü
- `/api/health` : Genel sistem durumu

## 10.3 Yedekleme Stratejisi

Supabase, veritabanının otomatik yedeklerini tutar:

- **Pro plan:** 7 günlük Point-in-Time Recovery (PITR) — herhangi bir saniyeye geri dönülebilir
- **Manuel yedek:** Supabase Dashboard > Settings > Backups > "Create Backup"
- **SQL dump (CLI):** `pg_dump -h <host> -U postgres <database> > backup_YYYY-MM-DD.sql`

Yedekler Flowra uygulama sunucusundan bağımsız tutulur; uygulama tamamen çökmüş olsa bile veriler kurtarılabilir.

---

*Bu belge Flowra v1.0 mimarisini tanımlar. Bileşen sürümleri ve konfigürasyon detayları için `.env.example` dosyasına ve `RELEASE_CERTIFICATION.md` belgesine bakınız.*